

Incident & On-Call Platform - ALL-IN Master Project Plan

This document is a complete, exhaustive, production-grade breakdown of the DevOps Incident & On-Call Platform Hackathon project. It is intentionally long, detailed, and uncompromising, designed to maximize scoring, cover every mandatory requirement, and unlock every available bonus.

Phase 0 - Strategic Foundation (Critical)

- Define clear Service-Oriented Architecture boundaries with zero shared business logic.
- Select technology stack optimized for speed and reliability (FastAPI, PostgreSQL, Docker).
- Design repository structure with strict separation of services, monitoring, CI/CD, and IaC.
- Prepare API versioning strategy (/api/v1) and consistent error handling conventions.

Phase 1 - Docker & Infrastructure as Code Backbone

- Single docker-compose.yml defining the entire platform stack.
- Shared Docker network for all services using service-name DNS resolution.
- Named volumes for PostgreSQL, Prometheus, and Grafana persistence.
- Health checks for every service and correct depends_on ordering.
- Multi-stage Dockerfiles, non-root execution, optimized image size (<500MB).

Phase 2 - Core Platform Services

- Alert Ingestion Service: schema validation, severity normalization, correlation logic.
- Incident Management Service: lifecycle tracking, MTTA and MTTR calculation.
- On-Call and Escalation Service: rotations, primary/secondary logic, timed escalation.
- Web UI: dashboards, incident details, acknowledge and resolve actions.

Phase 3 - Notification Service (Bonus)

- Mock email notifications via logs.
- Webhook notification support.
- Escalation notifications and delivery tracking.
- Metrics exposure for notification attempts and status.

Phase 4 - Observability Stack

- Prometheus scraping all services via /metrics endpoints.
- Grafana dashboards: Live Incident Overview and SRE Performance Metrics.
- Optional System Health Dashboard for service availability and resource usage.
- Log aggregation using Loki and Promtail.
- Distributed tracing using Jaeger across service calls.

Phase 5 - CI/CD Pipeline

- Local pipeline execution via shell script or Makefile.
- Stages: linting, unit testing, secret scanning, image build, vulnerability scan.
- Automated deployment using docker compose.
- Post-deployment health verification.
- Rollback strategy on failure.

Phase 6 - Security & Code Quality

- No hardcoded secrets; environment variables only.
- Pre-commit and CI secret scanning (GitLeaks / TruffleHog).
- Minimum 60% test coverage enforced.
- Linting and duplication checks.

Phase 7 - Documentation & Demo Preparation

- Comprehensive README with setup instructions and architecture overview.
- API documentation (OpenAPI).
- Architecture diagrams.
- Scripted demo flow from alert ingestion to resolution.
- Grafana and UI screenshots for judges.

Bonus Coverage Summary

- Notification Service implemented.
- Webhook notifications supported.
- Automated escalation workflows.
- Historical incident analytics.
- Centralized logging with Loki.

- Distributed tracing with Jaeger.
- Docker Compose scaling demonstration.

This master plan is designed to deliver a flawless, production-grade platform that demonstrates strong DevOps practices, solid SRE principles, and real-world operational maturity. Following this plan end-to-end positions the team for maximum possible score and top-tier ranking.